

Solutions for Homework Assignment #7

**Answer to Question 1.** We will denote this problem JSCHED, for “job scheduling”.

JSCHED is in NP: We prove this using the certificate-verifier characterization of NP. The certificate for a yes-instance of JSCHED is a list of  $k$  indices  $i_1, i_2, \dots, i_k$  in the range  $[1..n]$ . This has size polynomial in the size of the input. The verifier checks that the indices are pairwise distinct, that for all  $p$  in the list of indices in the certificate; that  $J_p$  does not contain overlapping intervals; and that for all  $p \neq q$  in the list of indices in the certificate, no two intervals of  $J_p$  and  $J_q$  overlap. This can be done deterministically in polynomial time.

JSCHED is NP-hard: We prove this by describing a polytime mapping reduction of the independent set problem IS to JSCHED.

Given an instance  $\langle G, k \rangle$  of IS, where  $G$  is an undirected graph and  $k$  is a positive integer, we show how to construct an instance  $\langle J_1, J_2, \dots, J_n, \ell \rangle$  of JSCHED so that

$$\begin{aligned} G \text{ has an independent set of size } k & \quad (\dagger) \\ \Leftrightarrow \text{ there are } \ell \text{ jobs among } J_1, J_2, \dots, J_n \text{ that have no intersecting intervals.} \end{aligned}$$

Let  $G = (V, E)$ , and let  $V = \{1, 2, \dots, n\}$ . For each node  $i \in V$  we create a job, denoted  $J_i$ , and for each edge  $\{i, j\} \in E$  we create an interval  $I_{\{i, j\}}$ . Note that, since  $\{i, j\}$  is the same edge as  $\{j, i\}$ , the intervals  $I_{\{i, j\}}$  and  $I_{\{j, i\}}$  are the same. The start and finish times of the intervals are not important, as long as they are chosen to make the intervals disjoint. For example, if the edges of  $G$  are  $e_1, e_2, \dots, e_m$ , we can assign interval  $[2t - 1, 2t]$  to the interval  $I_{e_t}$ . The intervals of job  $J_i$  are the intervals that correspond to the edges that are incident to node  $i$ ; that is,  $J_i = \{I_{\{i, j\}} : \{i, j\} \in E\}$ . The instance of JSCHED to which we map  $\langle G, k \rangle$  is  $\langle J_1, J_2, \dots, J_n, k \rangle$ .

The size of  $\langle J_1, J_2, \dots, J_n, k \rangle$  is clearly polynomial in the size of  $\langle G, k \rangle$ , and can be constructed from it in polynomial time.

By this construction, two distinct jobs  $J_i$  and  $J_j$  have intersecting intervals if and only if the corresponding nodes  $i$  and  $j$  of  $G$  have an edge connecting them. Therefore there is a set of  $k$  jobs that have no intersecting intervals if and only if the set of corresponding nodes have no edges between them, i.e., they form an independent set of  $k$  nodes. This proves  $(\dagger)$ , and completes the proof that  $\text{IS} \leq_m^p \text{JSCHED}$ .

**Answer to Question 2.**

a.

HP-DEC is in NP: We will prove this using the certificate-verifier characterization of NP. Given an instance  $\langle G, \ell, c, B, L \rangle$  of HP-DEC, the certificate is a list of nodes  $S = h_1, h_2, \dots, h_k$  (the villages where the hospitals will be built), and for each village  $u \in V$  a list of at most  $n$  nodes (intended to form a path from  $u$  to some node in  $S$ ). Each node can be represented using  $\lceil \log_2 n \rceil$  bits, where  $n = |V|$ , so the certificate is  $O(n^2 \log n)$  bits long, which is polynomial in the input size. The verifier checks that (a)  $\sum_{i=1}^k c(h_i) \leq B$  (the chosen villages satisfy the budget constraint), and (b) for each node  $u$ , the associated list of nodes forms a path from  $u$  to some node  $h_i$  in  $S$  and the length of this path is at most  $L$  (the chosen villages satisfy the proximity constraint). These can be done in  $O(n^2)$  time on a random access computer using the unit cost criterion, and therefore in polynomial time in the size of the input on a deterministic TM.

HP-DEC is NP-hard: It suffices to show that  $\text{HP-DEC}'$ , the special case of HP-DEC described in the hint, is NP-hard. We do so by describing a polytime mapping reduction of the vertex cover problem, VC, to  $\text{HP-DEC}'$ . Note that since the edge length function  $\ell$ , the cost function  $c$ , and the maximum distance  $L$

that are parts of the input to HP-DEC are now fixed, the input to HP-DEC' can be described as  $\langle G, B \rangle$ , where  $G$  is an undirected graph and  $B$  is a positive integer.

Given an instance  $\langle G, k \rangle$  of VC, where  $G = (V, E)$  is an undirected graph and  $k$  is a positive integer, we produce an instance  $\langle G', B \rangle$  of HP-DEC' so that

$$\begin{aligned} & G \text{ has a vertex cover of size } k \\ \Leftrightarrow & G' \text{ has a set of } B \text{ nodes so that every node is either in it or has a neighbour in it.} \end{aligned} \quad (\dagger)$$

The graph  $G'$  contains all the nodes and edges of  $G$  and, in addition, a new node  $n_e$  for each edge of  $G$ , connected to the two endpoints of  $e$ ; that is, if  $e = \{u, v\}$ ,  $G'$  has edges  $\{n_e, u\}$  and  $\{n_e, v\}$ . So, intuitively, every edge connecting  $u$  and  $v$  in  $G$  becomes a “triangle” consisting of  $u$ ,  $v$ , and  $n_{\{u,v\}}$  in  $G'$ . More precisely, if  $\langle G, k \rangle$  is the given instance of VC, we define graph  $G' = (V', E')$  and integer  $B$  as follows:

- $V' = V \cup \{n_e : e \in E\}$ ,
- $E' = E \cup \{\{n_e, u\} : e \in E \text{ and } u \in e \text{ (i.e., } u \text{ is an endpoint of edge } e)\}$ , and
- $B = k + |I|$ , where  $I$  is the set of isolated nodes of  $G$  (i.e., nodes not connected to any other node).

The size of  $\langle G', B \rangle$  is clearly polynomial in the size of  $\langle G, k \rangle$ , and can be produced from it in polynomial time. So the mapping  $\langle G, k \rangle \mapsto \langle G', B \rangle$  is polytime. It remains to prove  $(\dagger)$ .

ONLY IF: Suppose  $G$  has a vertex cover  $C$  of size  $k$ , and let  $I$  be the set of isolated nodes in  $G$ . Then every non-isolated node  $x$  of  $G'$  is either in  $C$  or a neighbour of a node in  $C$ : If  $x$  is one of the (non-isolated) nodes of  $G$  and is not in  $C$  then in fact all its neighbours are in  $C$  (otherwise, the edges connecting it to some of its neighbours would not be covered by  $C$ ); and if  $x$  is one of the new nodes, say  $n_{\{u,v\}}$ , then one of its neighbours  $u$  or  $v$  must be in  $C$  (otherwise,  $C$  does not cover the edge  $\{u, v\}$  of  $G$ ). Therefore, the set  $C \cup I$  is a set of at most  $k + |I|$  nodes so that every node of  $G'$  is either in it or has a neighbour in it, as wanted. (We need to include the isolated nodes explicitly, precisely because they have no neighbours.)

IF: Suppose that  $G'$  has a set  $D$  of at most  $B = k + |I|$  nodes such that every node of  $G'$  is in  $D$  or has a neighbour in  $D$ . Note that every isolated node in  $G$  must belong to  $D$ . Let  $D' = D - I$ .  $D'$  may contain some nodes that are nodes originally in  $G$  and some new nodes of the form  $n_{\{u,v\}}$ , where  $\{u, v\}$  is an edge of  $E$ . Let  $C$  be the set of nodes obtained from  $D'$  by replacing each new node  $n_{\{u,v\}}$  by one of its neighbours,  $u$  or  $v$ . Note that this has only nodes in  $G$ , and  $|C| \leq |D'|$  (it could be strictly smaller if we replace  $n_{\{u,v\}}$  by  $u$ , and  $u$  is already in  $D'$ ). So,  $|C| \leq |D'| = B - |I| = k$ . We claim that  $C$  is a vertex cover of  $G$ : Let  $\{u, v\}$  be an edge of  $G$ . Since  $u$  and  $v$  are the only neighbours of  $n_{\{u,v\}}$  in  $G'$ , at least one of these three must be in  $D'$  and therefore at least one of  $u$  and  $v$  must be in  $C$ . We have therefore shown that  $G$  has a vertex cover of size at most  $k$ , as wanted.

**b.** We will show a Cook reduction of HP-OPT to HP-DEC. Assume that we have a “black box” procedure HP-DEC-ORACLE( $\langle G, \ell, c, B, L \rangle$ ) that solves HP-DEC; i.e., it returns 1 if  $\langle G, \ell, c, B, L \rangle$  is a yes-instance of the the problem, and returns 0 otherwise.

Using HP-DEC-ORACLE and binary search we find the minimum budget  $B$  that is needed to satisfy the proximity constraint. Then, using HP-DEC-ORACLE again, we determine if it is possible to satisfy the proximity constraint after excluding a node (village)  $u$  from consideration as the possible location for a hospital. We can do this by setting the cost of building a hospital at  $u$  to  $B + 1$ , if it is not already higher than that, so that  $u$  could not be in a set of hospitals whose total cost is  $B$ . If excluding  $u$  still satisfies the proximity constraint with a budget  $B$  we ignore  $u$ ; otherwise, we place  $u$  in a set of nodes  $H$  (originally empty), without changing its cost. We continue by considering each node in this way. The nodes that are in  $H$  at the end of this process form a feasible set with minimum cost.

We describe the reduction algorithm in pseudocode below. If  $G$  is a graph,  $\hat{c}$  is a cost function on the nodes of  $G$ ,  $B$  is the minimum cost of a feasible set, and  $u$  is a node of  $G$ , we denote by  $\hat{c}^{-u}$  the cost function that assigns to node  $u$  the value  $\max(B + 1, c(u))$  and to every node other than  $u$  the same value as  $\hat{c}$ .

HP-OPT-SOLVER( $\langle G, \ell, c, L \rangle$ )

```

1   $minB := \min\{c(u) : u \in V\}$ 
2   $maxB := \sum_{u \in V} c(u)$ 
3  while  $minB \neq maxB$  do
4       $midB := \lfloor (minB + maxB)/2 \rfloor$ 
5      if HP-DEC-ORACLE( $\langle G, \ell, c, midB, L \rangle$ ) = 1 then  $maxB := midB$ 
6      else  $minB := midB + 1$ 
7  let  $u_1, u_2, \dots, u_n$  be the nodes of  $G$ 
8   $B := minB$  ▶  $B$  is the minimum budget needed to satisfy the proximity constraint
9   $\hat{c} := c$  ▶  $\hat{c}$  is the cost function  $c$  with the cost of eliminated nodes set to (at least)  $B + 1$ 
10  $H := \emptyset$  ▶  $H$  is the set of nodes in the minimum-cost feasible set identified so far
11 for  $i := 1$  to  $n$  do
12     if HP-DEC-ORACLE( $\langle G, \ell, \hat{c}^{-u_i}, B, L \rangle$ ) = 1 then  $\hat{c} := \hat{c}^{-u_i}$ 
13     else  $H := H \cup \{u_i\}$ 
14 return  $H$ 

```

This reduction algorithm is polynomial-time in the size of the input  $\langle G, \ell, c, L \rangle$  (if each call to HP-DEC-ORACLE counts as one step): The loop for the binary search in lines 3-6 is executed  $O(\log_2 \sum_{u \in V} c(u))$  times, which is polynomial in the size of the input, and each iteration also takes time polynomial in the size of the input (to compute the input to HP-DEC-ORACLE). The loop in lines 11-13 is executed  $n$  times, where  $n$  is the number of nodes of  $G$ , and each iteration takes time polynomial in the size of the input (again to compute the input to HP-DEC-ORACLE). So, the algorithm takes polynomial time on a random access computer under the unit-cost criterion, (assuming also unit cost for each call to the oracle) and so it also takes polynomial time on an oracle DTM.

Finally we prove the correctness of this algorithm. We call a set  $S$  of nodes **feasible** if it satisfies the proximity constraint. With a convenient abuse of notation, if  $\hat{c}: V \rightarrow \mathbb{Z}^+$  is a cost function, we define  $\hat{c}(S) = \sum_{u \in S} \hat{c}(u)$  — i.e.,  $\hat{c}(S)$  is the cost of building hospitals in the nodes (villages) in  $S$  under the cost function  $\hat{c}$ . So the algorithm must return a feasible set  $S$  that minimizes  $c(S)$ . Let  $H_i$  be the set of nodes in  $H$  at the end of the  $i$ -th iteration of the loop in lines 11-13. Since the algorithm outputs  $H_n$ , it remains to show that  $H_n$  is a feasible set that minimizes  $c(S)$ . This follows from the two observations below concerning the invariants satisfied by the loops in lines 3-6 and 11-13. (Both of these invariants can be shown by straightforward induction.)

- (1) The loop in lines 3-13 satisfies the invariant: “the minimum cost of a feasible set is between  $minB$  and  $maxB$ ”. Thus, the exit condition of this loop implies that when the loop terminates  $minB$  (and  $maxB$ ) is the minimum cost of a feasible set and therefore line 8 sets  $B$  to that value.
- (2) The loop in lines 11-13 satisfies the following invariant, where  $c_i$  denotes the cost function  $c$  at the end of the  $i$ -th iteration: “(a)  $\hat{c}_i$  is the same as the given cost function  $c$  except that the cost of every node in  $\{u_1, \dots, u_i\} - H_i$  is set to  $\max(B + 1, c(u))$ , and (b) there is a feasible set of nodes  $S$  such that  $H_i \subseteq S \subseteq H_i \cup \{u_j : j > i\}$  and  $\hat{c}_i(S) = B$ ”.

When the second loop terminates after iteration  $n$ , there are no nodes indexed higher than  $n$ , and so, by Invariant (2b), the set  $H_n$  returned by the algorithm is a feasible set  $S$  with  $\hat{c}_n(S) = B$ . By Invariant (2a)  $c(S) \leq \hat{c}_n(S) = B$ , where  $c$  is the given cost function  $c$ . But since  $S$  is feasible, by Invariant 1,  $c(S)$  cannot be smaller than  $B$ , so  $c(S) = B$ . Therefore the algorithm returns a feasible set with minimum cost, as wanted.

**THAT’S IT WITH HOMEWORK SOLUTIONS, FOLKS!**